EXPERIMENT 13

Case Study on the Limitations of Low-Code Platforms for a Complex Enterprise Use Case

---

Project: Insurance Claim Management System

Name	Pavithra Narayanan
Roll No	241401064
Department	CSBS

241401064

1. Aim

To study, through a case study, the limitations of low-code platforms in general — not only one product — when they are used for a complex enterprise use case: a global insurance organisation that processes claims across many countries, systems and user roles. The study also aims to recommend how the limitations can be reduced.

The case study shows where low-code gives clear benefit, where it begins to strain as complexity grows, and how a hybrid approach can keep the speed of low-code while avoiding its weak points.

2. Procedure

Step 1: Define the enterprise scenario — A realistic large-scale scenario was described: a global insurance company handling motor, health and property claims across many countries, with many employee roles, a legacy core policy system, partner and payment APIs, strict data-privacy and financial rules, and high claim volumes after major events. The Insurance Claim Management System built in Zoho Creator in the earlier experiments was used as the starting point.

Step 2: List the requirements — The scenario was broken into requirements such as scale, integration, custom logic, security, customisation and long-term maintenance (Section 3.2).

Step 3: Map the architecture — The scenario was drawn as an integration map, and each integration was marked as well supported, possible with effort, or strained for a typical low-code platform (Section 3.1).

Step 4: Identify limitations — Each requirement was examined for limitations that are common across low-code platforms, with the impact on this case and a mitigation (Section 3.3).

Step 5: Show how limits appear with growth — A conceptual chart and an indicative suitability chart were prepared to show where low-code fits well and where it strains (Sections 3.4 and 3.5).

Step 6: Decide and mitigate — A decision flow was prepared to choose between pure low-code, hybrid and custom development (Section 3.6), and a hybrid architecture was proposed (Section 3.7).

Note: this is an analytical case study. Specific limits, such as execution time, record volume or API calls, differ between platforms and plans, so none are quoted here; the current documentation of the chosen platform should be checked. The charts are conceptual or indicative, not measured benchmarks.

24140164

3. Output

3.1 Enterprise scenario and integration map

The diagram shows the scenario around a generic low-code platform. Vendor-native services and standard connectors are well supported (green). Payment gateways, partner APIs and regulatory needs are possible with effort (amber). The core policy system or mainframe and the high-volume fraud or AI decision engine are strained and usually need custom code (red).

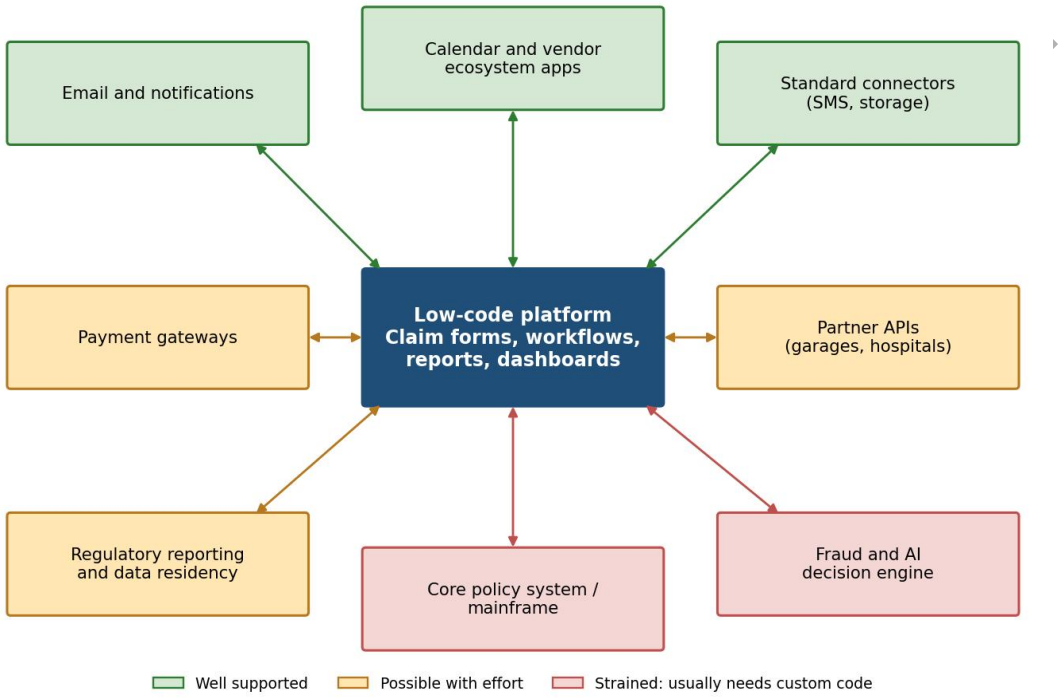


Figure 1: Enterprise Scenario and Integration Map (Conceptual Diagram)

3.2 Enterprise requirements

The requirements of the scenario are summarised below.

Requirement	Why it is complex
High volume and scale	Large numbers of users, claims and transactions, with peaks after major events such as floods or accidents.
Complex business rules	Country-specific rules, approval limits and settlement calculations that change often.
System integration	Legacy core policy system or mainframe, partner and payment APIs, identity and single sign-on.
Security and compliance	Data-privacy and financial regulations, audit trails, data residency.

Requirement	Why it is complex
Tailored experience	Different interfaces for policyholders, claims staff, surveyors and administrators.
Long-term maintenance	Years of change by many teams, and the ability to change platform later.

3.3 Limitation analysis

The limitations below are common across low-code platforms, with their impact on this case and a possible mitigation.

Area	Limitation of low-code	Impact on this case	Mitigation
Scalability and performance	Platform-managed runtime and shared resources; script execution and API usage are limited by platform and plan.	Claim surges and large reports may slow down or hit limits.	Efficient queries, avoid loops, move heavy jobs to external services.
Complex logic	Platform scripting suits business rules, not complex algorithms.	Fraud scoring, risk models and advanced settlement calculations are hard to express.	Call a custom microservice through an API.
Integration	Native connectors cover popular services; legacy systems and industry standards need custom work.	Core policy, mainframe and partner systems cannot be linked by simple connectors.	API gateway or middleware between the platform and legacy systems.
Customisation	Fixed UI components and platform conventions.	Highly tailored interfaces are limited.	Custom components or a separate front end over the platform API.
Security and compliance	Hosting, data residency and audit depth depend on the vendor.	Regulations may demand more control than the platform gives.	Review certifications, role-based access, keep sensitive data in a controlled system.
Governance and sprawl	Easy building leads to many unmanaged apps.	Duplicate, insecure or unsupported claim apps appear.	Central inventory, standards, centre of excellence.
Vendor lock-in	Apps, logic and workflows are tied to one platform and are hard to export.	A later move would mean rebuilding.	Keep core logic in APIs and data exportable.

4140164

3.4 Where low-code starts to strain

The conceptual chart shows that low-code gives a large benefit for simple needs, but as complexity, scale and integration demands grow its advantage falls, while custom code stays steady. The hybrid zone marks where combining both is usually best. The chart is a conceptual illustration, not measured data.

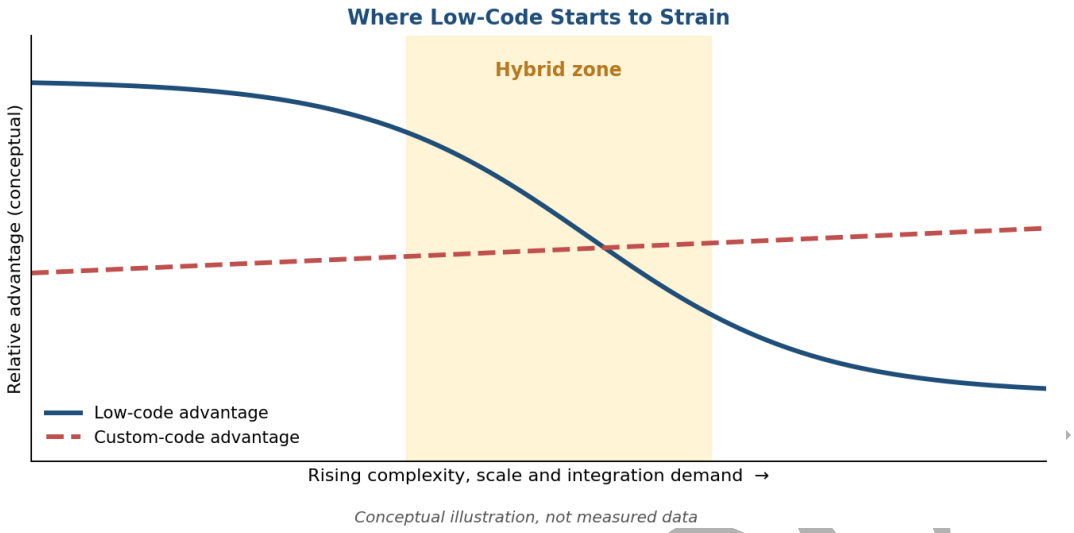


Figure 2: Where Low-Code Starts to Strain (Conceptual Chart)

3.5 Suitability of low-code by requirement

The chart rates how well low-code fits each requirement for this case. Rapid development and simple claim workflows score highest, while custom logic and portability score lowest. The values are indicative scores from this analysis, not benchmark results.

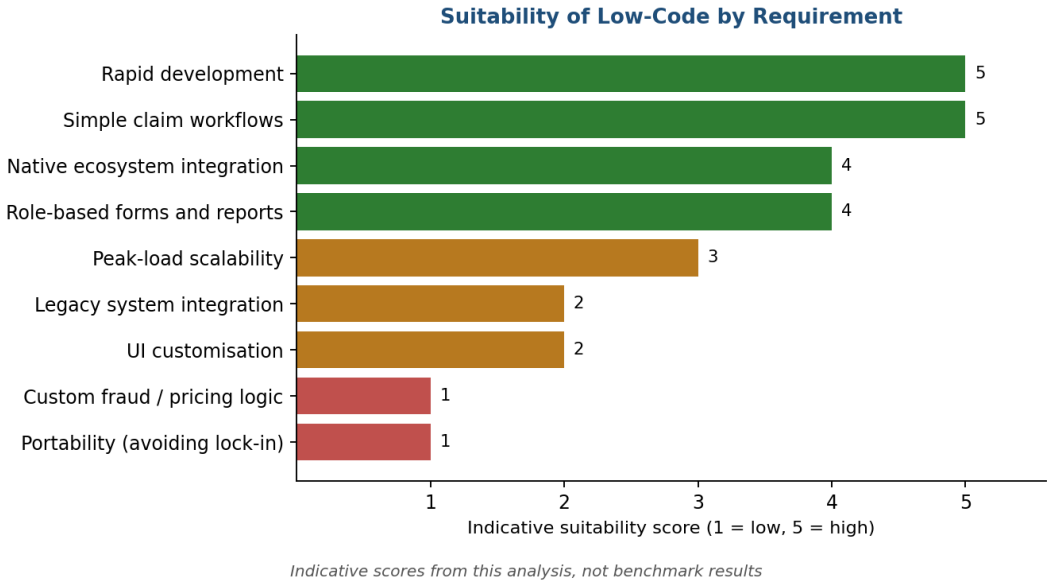


Figure 3: Suitability of Low-Code by Requirement (Indicative Scores)

3.6 Decision flow for choosing the approach

The decision flow asks whether the requirement is standard, whether it needs heavy computation or very high load, and whether it must integrate with legacy or strict-compliance systems. The

answers lead to pure low-code, custom code, a hybrid design, or low-code with connectors and functions.

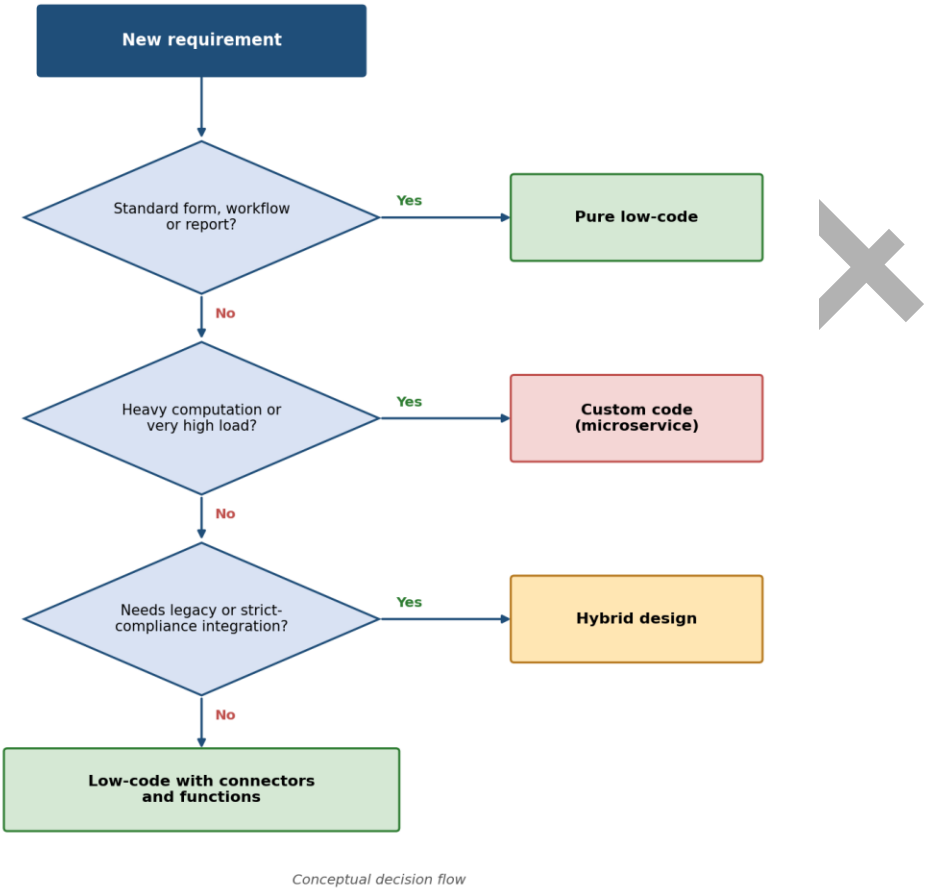


Figure 4: Decision Flow for Choosing the Approach (Conceptual Diagram)

3.7 Recommended hybrid architecture

The low-code platform keeps forms, workflows and the user interface and uses vendor-native services. An API gateway or middleware layer links it to custom microservices, the legacy core policy system, payment and partner systems, and a data warehouse for analytics and audit.

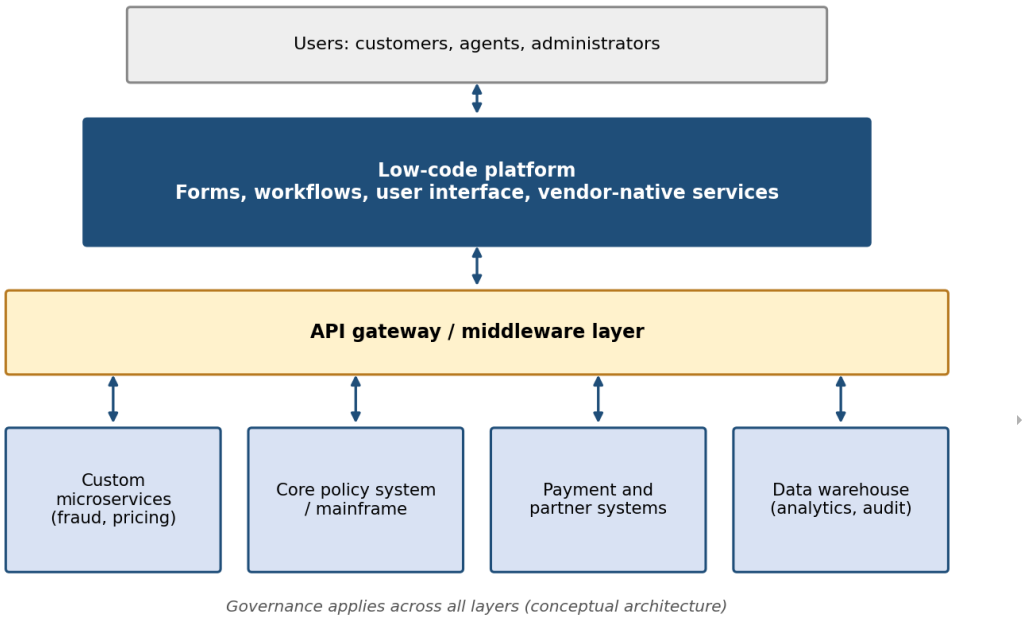


Figure 5: Recommended Hybrid Architecture (Conceptual Diagram)

4. Final Result

The case study showed that low-code is very effective for the standard parts of an enterprise claims system — forms, workflows, notifications and ecosystem integrations — and gives fast development with little code, whichever platform is used.

For a complex enterprise use case, the main limitations are scalability limits, complex custom logic, legacy and standards-based integration, restricted customisation, compliance control, governance sprawl and vendor lock-in. These were analysed in the limitation table, the conceptual chart and the suitability chart.

The recommended solution is a hybrid approach: keep low-code for forms, workflows and user interface, and add an API or middleware layer with custom services for heavy logic, legacy integration and analytics, under clear governance. No live testing or benchmark was carried out; the conclusions come from analysis of the case.